DOCKER-0 3

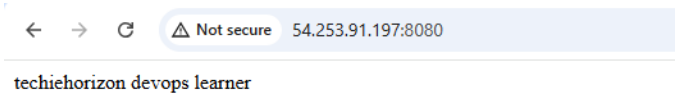
Task 1: Created a Docker Image from a Running Container

Objective: To create a new Docker image from a customized running container.

Steps performed:

1. Pulled the Nginx image from Docker Hub.
2. Created and started an Nginx container.
3. Customized the default web page using a custom message.
4. Verified the changes using the `curl` command.
5. Created a new Docker image using `docker commit`.
6. Verified the image using `docker images`.

```
root@ip-172-31-47-117:~# docker images
                                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker03-nginx:v1    00a55bad83f8        241MB         63.9MB
nginx:latest         332d0beea97c        243MB         66.8MB    U
root@ip-172-31-47-117:~# docker rename kind_einstein nginx
root@ip-172-31-47-117:~# ls
snap
root@ip-172-31-47-117:~# docker images
                                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker03-nginx:v1    00a55bad83f8        241MB         63.9MB
nginx:latest         332d0beea97c        243MB         66.8MB    U
root@ip-172-31-47-117:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS
                    NAMES
c9703eee6b27   nginx     "/docker-entrypoint.…"   16 minutes ago   Up 16 minutes   0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp   nginx
root@ip-172-31-47-117:~#
```



Task 2 — Short documentation notes

Objective: To transfer a Docker image between EC2 instances without using a Docker registry.

Steps performed:

1. Exported the custom image `docker03-nginx:v1` into a TAR file using `docker save`.
2. Copied the TAR file from the source EC2 to the Windows computer using `scp`.
3. Transferred the TAR file from Windows to the destination Ubuntu EC2.

4. Loaded the image on the destination using `docker load`.
5. Verified the imported image using `docker images`.

Result: Successfully transferred and loaded the Docker image on the destination EC2 instance.

```
Admin@360-2025 MINGW64 ~
$ scp -i "/c/Users/Admin/Downloads/docker.pem" docker03-nginx.tar ubuntu@3.107.83.138:/home/ubuntu/
The authenticity of host '3.107.83.138 (3.107.83.138)' can't be established.
ED25519 key fingerprint is: SHA256:n5/2P+mdbkk9ZwYXbHOmgghs13uwIksfG5hAZ+8my5Y
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.107.83.138' (ED25519) to the list of known hosts.
docker03-nginx.tar                                    100%   61MB   1.6MB/s   00:37
```

```
root@ip-172-31-34-86:~# scp -i "/c/Users/Admin/Downloads/docker.pem" docker03-nginx.tar ubuntu@3.107.83.138:/home/ubuntu/
scp: stat local "docker03-nginx.tar": No such file or directory
root@ip-172-31-34-86:~# sudo docker load -i /home/ubuntu/docker03-nginx.tar
Loaded image: docker03-nginx:v1
root@ip-172-31-34-86:~# sudo docker images
IMAGE              ID            DISK USAGE   CONTENT SIZE   EXTRA
docker03-nginx:v1  00a55bad83f8        241MB         63.9MB
root@ip-172-31-34-86:~#
```

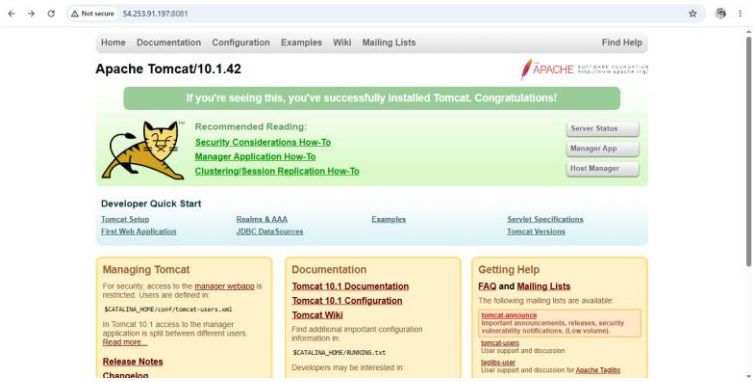
Task 3: Create a Custom Tomcat Image Using Alpine Linux

Objective: To create a lightweight Docker image using Alpine Linux and install and run Apache Tomcat inside it.

Steps performed:

1. Created a project directory and configured a Dockerfile.
2. Used Alpine Linux as the base image.
3. Installed OpenJDK 17 and required packages.
4. Downloaded and configured Apache Tomcat.
5. Built the custom image `alpine-tomcat:v1`.
6. Ran a container and exposed Tomcat on port `8081`.
7. Verified that Tomcat was running successfully.

```
 ---> c6d4c3524d81
Successfully built c6d4c3524d81
Successfully tagged alpine-tomcat:v1
root@ip-172-31-47-117:~/alpine-tomcat# sudo docker run -d --name alpine-tomcat -p 8081:8080 alpine-tomc
at:v1
4cac07e670c39cf3ee4363b6c4a553462b657f30c44f14f7aa2fce86f37fb53f
root@ip-172-31-47-117:~/alpine-tomcat# sudo docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED              STATUS              PORTS
                                  NAMES
4cac07e670c3   alpine-tomcat:v1   "catalina.sh run"        43 seconds ago       Up 42 seconds       0.0.0
.0:8081->8080/tcp, [::]:8081->8080/tcp   alpine-tomcat
c9703eee6b27   nginx              "/docker-entrypoint.…"   About an hour ago    Up About an hour    0.0.0
.0:8080->80/tcp, [::]:8080->80/tcp       nginx
root@ip-172-31-47-117:~/alpine-tomcat# curl -I http://localhost:8081
HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Date: Fri, 16 Oct 2026 11:44:35 GMT
```



Result: Successfully built a custom Alpine-based Tomcat Docker image and verified that the Tomcat server was running.

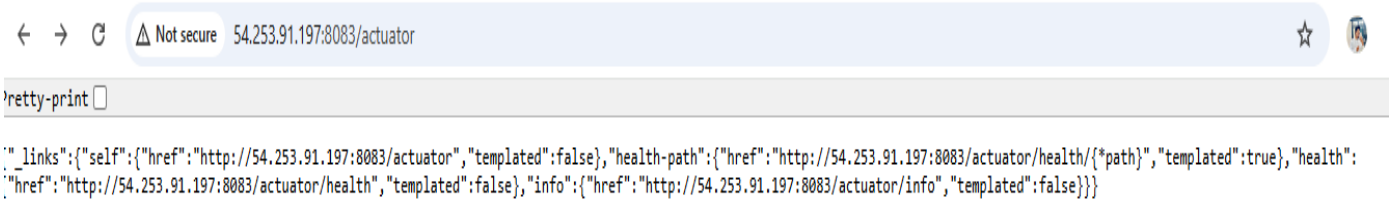
Task 4: Single-Stage and Multi-Stage Dockerfiles

Objective: To build and run a Java Spring Boot application using single-stage and multi-stage Docker builds.

Steps Completed

1. Cloned the source code from `https://github.com/betawins/multi-stage-example.git`.
2. Created a single-stage Dockerfile using Maven and Java 8.
3. Created a multi-stage Dockerfile with separate build and runtime stages.
4. Built the Docker images `example-single:v1` and `example-multi:v1`.
5. Ran both containers with port mappings `8082:8080` and `8083:8080`.
6. Configured AWS EC2 security group inbound rules for the application ports.
7. Verified the multi-stage application in the browser using the Spring Boot Actuator endpoint.

```
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25.023 s
[INFO] Finished at: 2026-10-16T12:47:15Z
[INFO] ------------------------------------------------------------------------
 ---> Removed intermediate container 54435f67e2ae
 ---> 3042b90016f7
Step 5/6 : EXPOSE 8082
 ---> Running in b13e1d09fe18
 ---> Removed intermediate container b13e1d09fe18
 ---> f55ed1873bfa
Step 6/6 : ENTRYPOINT ["java", "-jar", "target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
 ---> Running in ac8cef26a1c4
 ---> Removed intermediate container ac8cef26a1c4
 ---> 56177a2747b0
Successfully built 56177a2747b0
Successfully tagged example-single:v1
root@ip-172-31-47-117:~/multi-stage-example# docker images
                                                                    i Info →  U  In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine-tomcat:v1       c6d4c3524d81       314MB         85.6MB    U
alpine:3.21            ce64758a109e      12.2MB         3.71MB
docker03-nginx:v1      00a55bad83f8       241MB         63.9MB
example-single:v1      56177a2747b0       964MB          307MB
maven:3.8.6-openjdk-8  ff18d86faefa       823MB          246MB
nginx:latest           332d0beea97c       243MB         66.8MB    U
```




```
"_links":{"self":{"href":"http://54.253.91.197:8083/actuator","templated":false},"health-path":{"href":"http://54.253.91.197:8083/actuator/health/{*path}","templated":true},"health":
"href":"http://54.253.91.197:8083/actuator/health","templated":false},"info":{"href":"http://54.253.91.197:8083/actuator/info","templated":false}}}
```

multi stage docker file:-

Name:example multi:v1 `image id {5299e859aae4}`

```
Status: Downloaded newer image for eclipse-temurin:8-jre
 ---> 5c9a04052cf8
Step 6/9 : WORKDIR /app
 ---> Running in 751feff3c2c5
 ---> Removed intermediate container 751feff3c2c5
 ---> ab7d7f6b46c1
Step 7/9 : COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar
 ---> 9018f6a86097
Step 8/9 : EXPOSE 8082
 ---> Running in 685abc04bc0d
 ---> Removed intermediate container 685abc04bc0d
 ---> 73ea95a6f212
Step 9/9 : ENTRYPOINT ["java", "-jar", "app.jar"]
 ---> Running in a7395c08827c
 ---> Removed intermediate container a7395c08827c
 ---> 5299e859aae4
Successfully built 5299e859aae4
Successfully tagged example-multi:v1
root@ip-172-31-47-117:~/multi-stage-example# docker images
                                                                                    i Info →  U  In Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine-tomcat:v1         c6d4c3524d81       314MB         85.6MB    U
alpine:3.21              ce64758a109e      12.2MB         3.71MB
docker03-nginx:v1        00a55bad83f8       241MB         63.9MB
eclipse-temurin:8-jre    5c9a04052cf8       405MB          112MB
example-multi:v1         5299e859aae4       437MB          124MB
```

Task 5: Install Docker Compose and Execute a Sample Application

Docker Compose is used to run and manage one or more Docker containers using a single YAML configuration file instead of typing separate Docker commands for each container.

Objective: To deploy and manage a sample application using Docker Compose.

1. Verified Docker installation on the Ubuntu EC2 instance.
2. Created a `docker-compose.yml` file with the Nginx image.
3. Configured port mapping `8084:80`.
4. Started the application using `docker compose up -d`.
5. Verified the running container using `docker compose ps`.
6. Tested Nginx using `curl http://localhost:8084` and received the welcome page.

```
root@ip-172-31-47-117:~/docker-compose# docker compose up -d
[+] Running 2/2
 ✔ Network docker-compose_default  Created                                          0.1s
 ✔ Container compose-nginx         Started                                          1.6s
root@ip-172-31-47-117:~/docker-compose# docker-compose ps
docker-compose: command not found
root@ip-172-31-47-117:~/docker-compose# docker compose ps
NAME            IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORT
S
compose-nginx   nginx:latest   "/docker-entrypoint.…"   web       40 seconds ago   Up 39 seconds   0.0.
0.0:8084->80/tcp, [::]:8084->80/tcp
root@ip-172-31-47-117:~/docker-compose# curl http://localhost:8084
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>
```

Task 6: Scan a Docker Image When It Is Pushed to a Docker Registry

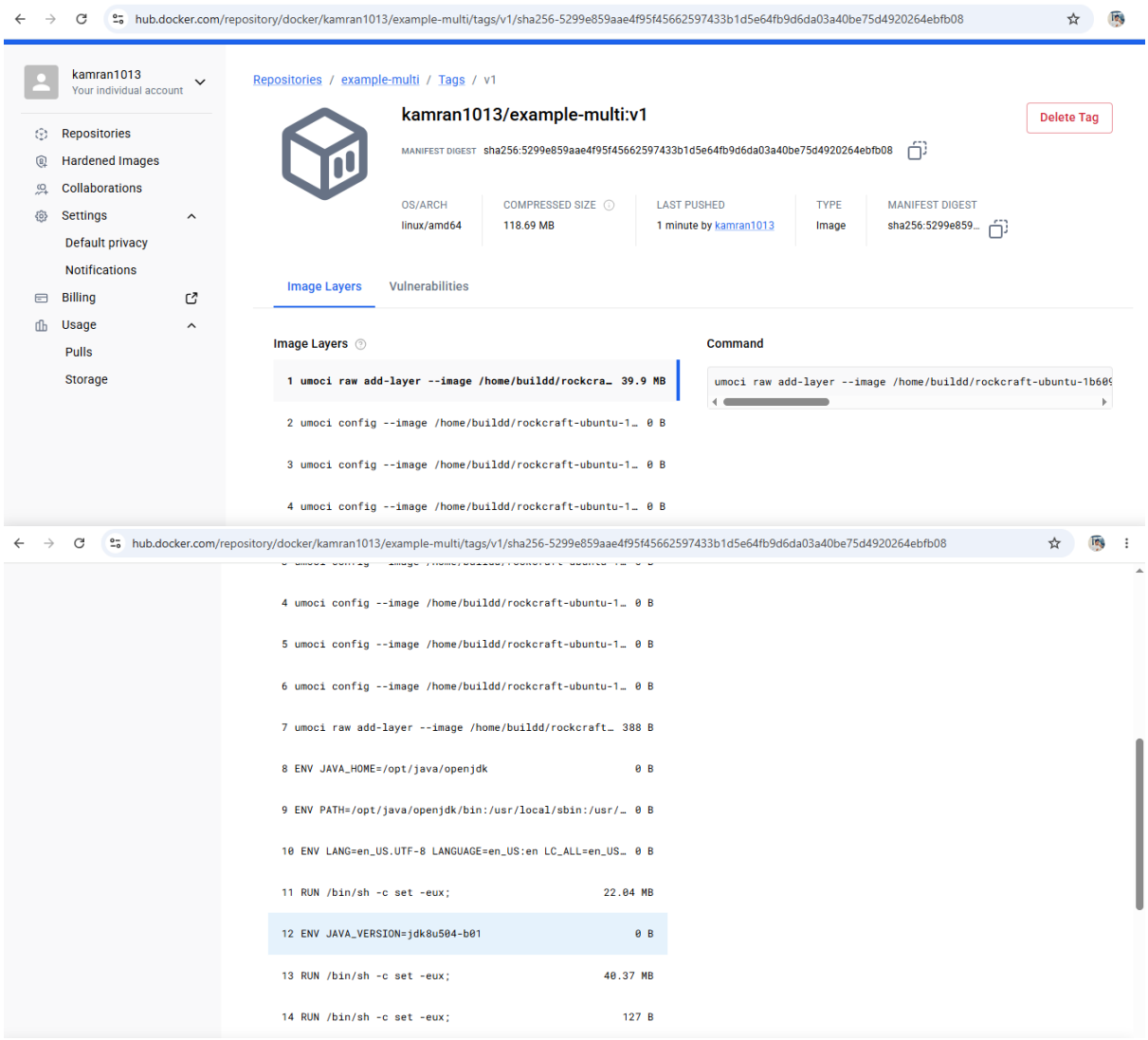
Understand the task:

When you push a Docker image to a registry such as Docker Hub, you want to check whether it contains known security vulnerabilities.

Steps performed

1. Installed Trivy on the Ubuntu EC2 instance.
2. Increased the EBS root volume to 20 GB and expanded the root partition to resolve disk-space issues.
3. Configured Trivy to use `/var/tmp` for temporary files during scanning.
4. Scanned the Docker image `example-multi:v1` for security vulnerabilities.
5. Reviewed the scan results, including reported CVEs.
6. Created a Docker Hub access token with Read and Write permissions.
7. Logged in to Docker Hub successfully.
8. Tagged the image as `kamran1013/example-multi:v1`.
9. Pushed the image to the Docker Hub repository successfully.
10. Verified the pushed image by scanning `kamran1013/example-multi:v1` with Trivy.





Result

Successfully scanned the Docker image using Trivy and pushed it to Docker Hub. The scan identified reported vulnerabilities for security review.